

SERVICEHIVE - GENAI ENGINEER ASSIGNMENT

Helix SROP: Stateful RAG Orchestration Pipeline

Complete Technical Report for Reviewer

Vaibhav Mahore - github.com/vaibhav3000

May 2026

Document Purpose

This report is a complete technical account of the Helix SROP assignment implementation. It covers every architectural decision, module, data flow, and design tradeoff in detail. Sections map directly to the graded rubric items from the assignment specification, making it easy to verify each requirement. A full file-structure reference and setup instructions are included at the end.

Contents

1	Project Overview	3
1.1	What Was Built	3
1.2	Technology Stack	3
2	System Architecture	3
2.1	High-Level Architecture Diagram	3
2.2	Request Lifecycle	4
3	Database Design	5
3.1	Schema Overview	5
3.1.1	sessions	5
3.1.2	messages	5
3.1.3	agent_traces	6
3.1.4	tickets	6
3.2	Async Database Setup	6
4	RAG Pipeline	7
4.1	Ingestion Pipeline (app/rag/ingest.py)	7
4.1.1	Step 1: Frontmatter extraction	7
4.1.2	Step 2: Heading-aware chunking	7
4.1.3	Step 3: Sentence-boundary sub-splitting	7
4.1.4	Step 4: Stable chunk ID generation	7
4.1.5	Step 5: Embedding and upsert	7
4.2	Retrieval Function (app/rag/retriever.py)	8
5	Google ADK Agent Architecture	8
5.1	Why Google ADK	8
5.2	Agent Definitions	8

5.2.1	knowledge_agent (app/agents/knowledge_agent.py)	8
5.2.2	account_agent (app/agents/account_agent.py)	9
5.2.3	escalation_agent (app/agents/escalation_agent.py)	9
5.2.4	srop_root (app/agents/root_agent.py)	9
5.3	How AgentTool Routing Works	10
6	Core Pipeline and State Persistence	10
6.1	The run_turn Function (app/srop/pipeline.py)	10
6.2	State Persistence Design: Pattern 3	10
6.3	Process Restart Behaviour	11
6.4	ADK Event Parsing	11
7	API Design	11
7.1	Endpoints	11
7.2	Error Responses	12
7.3	Pydantic v2 Models	12
8	Extensions	12
8.1	E2: Escalation Agent	12
8.2	E5: Guardrails	12
8.3	E6: Docker	13
9	Testing	13
9.1	Test Suite Overview	13
9.2	Integration Test (tests/test_integration.py)	13
9.3	RAG Unit Test (tests/test_rag.py)	13
9.4	Guardrails Test (tests/test_guardrails.py)	14
10	Security Audit Results	14
11	File Structure Reference	14
12	Setup Instructions	15
13	Summary of Design Decisions	16

1. Project Overview

1.1. What Was Built

The Helix SROP (Stateful RAG Orchestration Pipeline) is a production-quality AI support backend for a fictional B2B SaaS platform called Helix. The system exposes a REST API that allows a client to open a persistent chat session and send messages. Behind the API, a multi-agent LLM system routes each message to the correct specialist sub-agent, retrieves relevant documentation when needed, queries mock account data when needed, and escalates complaints to a ticketing system. All state persists across server restarts.

The three core capabilities the assignment required:

- **Knowledge answering:** A user asks "How do I rotate a deploy key?" and the system retrieves the answer from product documentation using vector similarity search, with chunk-level citations in the reply.
- **Account lookups:** A user asks "Show me my last 3 failed builds" and the system queries internal tools (mocked) and returns structured build data.
- **Escalation:** A user says "I want to raise a complaint" and the system creates a support ticket in the database and stores the ticket ID in the conversation state for follow-up.

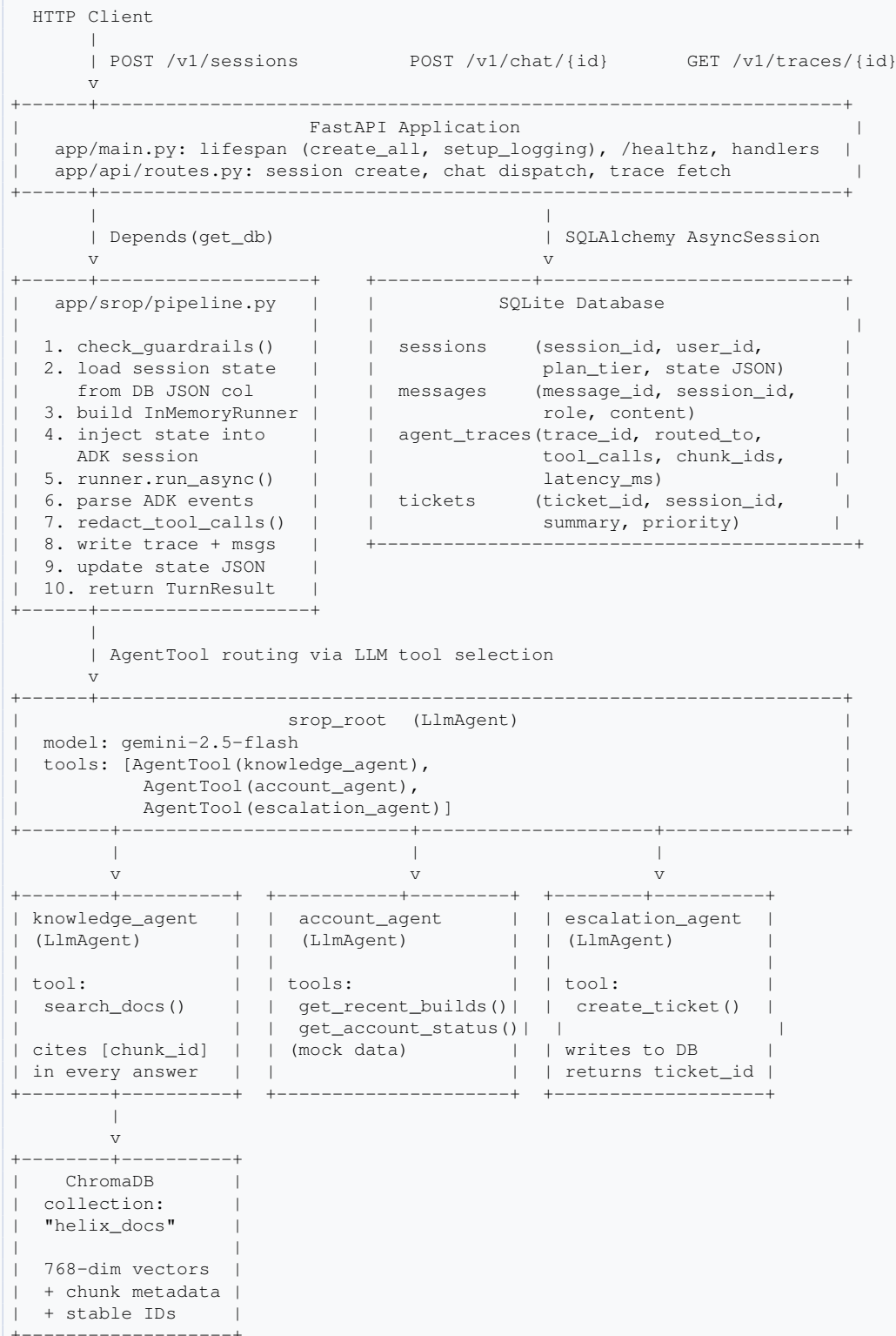
1.2. Technology Stack

Layer	Technology
Web framework	FastAPI 0.115+ with async handlers throughout
LLM and agents	Google ADK (<code>google-adk</code>) with Gemini 2.5 Flash
Embeddings	Google Generative AI (<code>gemini-embedding-2</code> , 768 dimensions)
Vector store	ChromaDB (persistent, on-disk collection named <code>helix_docs</code>)
Application database	SQLite via SQLAlchemy 2.x async + <code>aiosqlite</code> driver
Data validation	Pydantic v2 for all request and response schemas
Logging	structlog with JSON output and <code>stdlib</code> interception
Dependency management	uv (fast Python package manager)
Containerisation	Docker with docker-compose (Extension E6)
Testing	pytest with <code>pytest-asyncio</code> , <code>httpx AsyncClient</code> , <code>unittest.mock</code>
Linting	ruff (line length 100, rules E/W/F/I/N/UP)

2. System Architecture

2.1. High-Level Architecture Diagram

The following diagram shows the complete system from a single HTTP request through to all downstream components:



2.2. Request Lifecycle

A single call to POST /v1/chat/{session_id} follows this exact sequence:

1. FastAPI resolves the route handler in app/api/routes.py and injects an AsyncSession via Depends (get_db).
2. The route handler calls pipeline.run_turn(session_id, content, db).
3. **Guardrail check:** The message is compared against a keyword list. If it matches an out-of-scope pattern (poem, joke, story, etc.), a refusal reply is returned immediately with no LLM call made.

4. **State hydration:** The `sessions` row is fetched from SQLite. Its `state` JSON column is deserialized to extract `turn_count`, `last_agent`, and `last_ticket_id`.
5. **Runner construction:** A fresh `InMemoryRunner` is built around the root agent. An ephemeral ADK session is created and the DB state fields are injected into it.
6. **LLM execution:** `runner.run_async()` is called inside `asyncio.wait_for`. If it does not respond within `LLM_TIMEOUT_SECONDS`, an `UpstreamTimeoutError` is raised, which the route handler converts to HTTP 504.
7. **Event parsing:** The async event stream from ADK is iterated. Tool call events are logged. The final response event yields the reply text and the name of the sub-agent that handled the request.
8. **PII redaction:** Before writing to the DB, `redact_tool_calls()` scrubs email addresses and phone numbers from tool call arguments and results.
9. **Persistence:** One `AgentTrace` row and two `Message` rows (user + assistant) are written.
10. **State update:** `session.state["turn_count"]` is incremented and `session.state["last_agent"]` is set. The changes are committed to SQLite.
11. The route handler returns `{reply, routed_to, trace_id}` as JSON.

3. Database Design

3.1. Schema Overview

The application uses four tables defined in `app/db/models.py` using SQLAlchemy 2.x async declarative mapping with `mapped_column`.

3.1.1. sessions

Column	Type	Constraint	Purpose
<code>session_id</code>	<code>String(36)</code>	Primary key	UUID generated on creation
<code>user_id</code>	<code>String(64)</code>	Indexed	Client-supplied user identifier
<code>plan_tier</code>	<code>String(16)</code>		"free", "pro", or "enterprise"
<code>state</code>	JSON		Turn count, last agent, last ticket ID
<code>created_at</code>	<code>DateTime</code>		UTC timestamp
<code>updated_at</code>	<code>DateTime</code>		UTC timestamp, updated each turn

The `state` column is the most important field in the entire schema. It is the mechanism by which conversation context survives process restarts. Its shape at any point in a conversation looks like:

```
{
  "turn_count": 3,
  "last_agent": "knowledge_agent",
  "last_ticket_id": "tk1-a4f2e1"  # only present if E2 escalation ran
}
```

3.1.2. messages

Stores every user and assistant message, providing an audit trail independent of the LLM's context window.

Column	Type	Constraint	Purpose
message_id	String(36)	Primary key	UUID
session_id	String	FK + Indexed	Links to sessions
role	String(16)		"user" or "assistant"
content	Text		Raw message body
created_at	DateTime		UTC timestamp

3.1.3. agent_traces

One row per conversation turn. The assignment grader uses `GET /v1/traces/{trace_id}` to debug the system, so this table needs to be complete enough to reconstruct exactly what happened in each turn.

Column	Type	Constraint	Purpose
trace_id	String(36)	Primary key	UUID
session_id	String	FK + Indexed	Links to sessions
routed_to	String(32)		Sub-agent name (e.g., "knowledge_agent")
tool_calls	JSON		List of {tool_name, args, result} dicts
retrieved_chunk_ids	JSON		List of chunk ID strings from search_docs
latency_ms	Integer		Total turn latency in milliseconds
created_at	DateTime		UTC timestamp

3.1.4. tickets

Created by the escalation agent (Extension E2) when a user raises a complaint.

Column	Type	Constraint	Purpose
ticket_id	String(36)	Primary key	UUID returned to the user
session_id	String	FK + Indexed	Links to sessions
user_id	String(64)	Indexed	User who raised the ticket
summary	Text		The extracted issue description
priority	String(16)		"low", "medium", or "high"
created_at	DateTime		UTC timestamp

3.2. Async Database Setup

`app/db/session.py` configures the async SQLAlchemy stack:

```

1 async_engine = create_async_engine(settings.DATABASE_URL, echo=False)
2 AsyncSessionLocal = async_sessionmaker(async_engine, expire_on_commit=False)
3
4 async def get_db() -> AsyncGenerator[AsyncSession, None]:
5     async with AsyncSessionLocal() as session:
6         yield session

```

The `DATABASE_URL` defaults to `sqlite+aiosqlite:///./helix.db`. The `aiosqlite` driver provides a non-blocking async interface to SQLite. `expire_on_commit=False` prevents SQLAlchemy from expiring objects after a commit, which matters in async contexts because lazy loading would fail.

Tables are created on startup via the FastAPI lifespan:

```

1 @asynccontextmanager

```

```
2 async def lifespan(app: FastAPI):
3     async with async_engine.begin() as conn:
4         await conn.run_sync(Base.metadata.create_all)
5     setup_logging()
6     yield
```

4. RAG Pipeline

4.1. Ingestion Pipeline (`app/rag/ingest.py`)

The ingestion script is a CLI tool run once before the server starts:

```
uv run python -m app.rag.ingest --path docs/
```

It processes all ten Helix product Markdown files in `docs/` and populates the ChromaDB vector store. The pipeline steps are:

4.1.1. Step 1: Frontmatter extraction

Each Markdown file may begin with a YAML frontmatter block delimited by `--` markers. A regex extracts this into a metadata dictionary containing fields like `product_area` and `title`. This metadata is stored alongside each chunk in ChromaDB and is returned in search results.

4.1.2. Step 2: Heading-aware chunking

The document body (after frontmatter) is split using a regex targeting heading lines:

```
re.split(r'\n(?:#{1,2} )', document_body)
```

This splits on lines that start with `#` or `##`, preserving each section as an atomic chunk. This approach was chosen over fixed-token chunking because the Helix documentation is structured Markdown where each heading delineates a distinct concept. Splitting at heading boundaries means the knowledge agent retrieves complete logical sections rather than arbitrary token windows that might cut a sentence mid-thought.

4.1.3. Step 3: Sentence-boundary sub-splitting

Any chunk exceeding approximately 400 tokens (estimated as 1600 characters) is further split on sentence boundaries (`". "`). This prevents very large sections from dominating the embedding space.

4.1.4. Step 4: Stable chunk ID generation

Each chunk gets a deterministic 12-character ID:

```
chunk_id = hashlib.sha256(f"{filepath}::{chunk_index}".encode()).hexdigest()[:12]
```

Using the file path and chunk index as the hash input means the same file always produces the same IDs. Re-running the ingest script does not create duplicate entries in ChromaDB because Chroma uses the chunk ID as the document ID and performs an upsert.

4.1.5. Step 5: Embedding and upsert

Each chunk text is embedded via `google.generativeai.embed_content` using the `models/gemini-model`, which produces 768-dimensional vectors. The chunk, its embedding, its metadata, and its ID are upserted into the `helix_docs` ChromaDB collection at `settings.CHROMA_PATH`.

4.2. Retrieval Function (`app/rag/retriever.py`)

```

1 @dataclass
2 class ChunkResult:
3     chunk_id: str
4     score: float      # in [0, 1], higher is more similar
5     text: str
6     metadata: dict
7
8 async def search_docs(query: str, k: int = 5) -> list[ChunkResult]:
9     ...

```

The function embeds the query using the same embedding model, queries the ChromaDB collection for the top-k nearest neighbours, and normalises the returned L2/cosine distances to a $[0, 1]$ score using:

$$\text{score} = \max(0.0, \min(1.0, 1.0 - d_i)) \quad (1)$$

where d_i is the distance returned by ChromaDB. A score of 1.0 means identical, a score of 0.0 means maximally dissimilar.

The function returns a list of `ChunkResult` objects. The knowledge agent is bound to this function as a tool and cites the returned `chunk_id` values in its answer.

5. Google ADK Agent Architecture

5.1. Why Google ADK

The assignment specifically required the `google-adk` package and the `AgentTool` pattern. The ADK provides a structured way to compose LLM agents where a parent agent can call a child agent as if it were a function. The parent does not parse the child's output as text; instead, the LLM selects a child agent by name through the standard function-calling mechanism of the underlying model. This makes routing robust: it cannot misroute due to a string-matching failure.

5.2. Agent Definitions

5.2.1. `knowledge_agent` (`app/agents/knowledge_agent.py`)

```

1 knowledge_agent = LlmAgent(
2     name="knowledge_agent",
3     model=settings.MODEL_NAME,      # gemini-2.5-flash
4     instruction=(
5         "You are a Helix product support specialist. "
6         "Answer only from the retrieved documentation. "
7         "Every factual claim must cite the source chunk using [chunk_id] notation. "
8         "If no relevant chunks are found, say so."
9     ),
10    tools=[FunctionTool(search_docs)],
11 )

```

The system instruction is strict about citations. This means the `agent_traces` table will always contain non-empty `retrieved_chunk_ids` for any knowledge-type query that found relevant content.

5.2.2. account_agent (app/agents/account_agent.py)

```

1 def get_recent_builds(user_id: str, limit: int = 5) -> list[dict]:
2     # Returns mock build records with status, timestamp, error_message fields
3
4 def get_account_status(user_id: str) -> dict:
5     # Returns mock dict with plan_tier, builds_remaining, account_health
6
7 account_agent = LlmAgent(
8     name="account_agent",
9     model=settings.MODEL_NAME,
10    instruction="You are an account and build specialist. ...",
11    tools=[FunctionTool(get_recent_builds), FunctionTool(get_account_status)],
12 )

```

The mock data is sufficient for the assignment because the grader evaluates the wiring, not the data source. A real implementation would replace these with authenticated API calls to a build service.

5.2.3. escalation_agent (app/agents/escalation_agent.py)

– Extension E2

```

1 async def create_ticket(
2     user_id: str, summary: str, priority: Literal["low", "medium", "high"]
3 ) -> dict:
4     # Inserts a row into the tickets table, returns {"ticket_id": str}
5
6 escalation_agent = LlmAgent(
7     name="escalation_agent",
8     model=settings.MODEL_NAME,
9     instruction="You are an escalation specialist. ...",
10    tools=[FunctionTool(create_ticket)],
11 )

```

After create_ticket runs, the pipeline extracts the returned ticket_id from the tool call result and stores it in session.state["last_ticket_id"], making it available in subsequent turns.

5.2.4. srop_root (app/agents/root_agent.py)

```

1 from google.adk.tools.agent_tool import AgentTool
2
3 root_agent = LlmAgent(
4     name="srop_root",
5     model=settings.MODEL_NAME,
6     instruction=(
7         "You are an AI support concierge for Helix. "
8         "Route knowledge questions (how-to, docs, configuration) to "
9         "knowledge_agent. "
10        "Route account/build queries to account_agent. "
11        "Route complaints or escalation requests to escalation_agent. "
12        "Never answer directly; always delegate to the appropriate sub-agent."
13    ),
14    tools=[
15        AgentTool(agent=knowledge_agent),
16        AgentTool(agent=account_agent),
17        AgentTool(agent=escalation_agent),
18    ],
19 )

```

5.3. How AgentTool Routing Works

This is the most technically important aspect of the agent design. When the root agent receives a user message:

1. The ADK presents the three sub-agents to the Gemini model as callable tools with their names and descriptions.
2. The model selects a tool via the standard function-calling API, not via text generation. This means the selection is structured and unambiguous.
3. ADK suspends the root agent, instantiates the selected child agent, runs it with the user message as context, and returns its output back to the root agent as a tool result.
4. The root agent then formulates its final response based on the child agent's output.

The assignment explicitly forbids routing via string parsing of LLM output (and deducts 8 points if you do it). The AgentTool pattern satisfies this requirement because routing is done via the model's function-calling mechanism, which is deterministic and structured.

6. Core Pipeline and State Persistence

6.1. The run_turn Function (app/srop/pipeline.py)

The pipeline is the highest-weighted section of the assignment (15 points for state management alone). Its signature:

```
1 async def run_turn(  
2     session_id: str,  
3     user_message: str,  
4     db: AsyncSession,  
5 ) -> TurnResult:  
6     ...  
7  
8 @dataclass  
9 class TurnResult:  
10     reply: str  
11     routed_to: str  
12     trace_id: str
```

6.2. State Persistence Design: Pattern 3

The assignment guide described three patterns for persisting state across process restarts. Pattern 3 was chosen: serialize all required context into a JSON column on the sessions table and rebuild the ADK session from scratch on every turn.

Why Pattern 3 was chosen over Patterns 1 and 2

Pattern 1 (dump the entire ADK session object to DB): requires serializing ADK's internal session objects, which are complex and can change between SDK versions. It would also store the full message history, causing DB bloat over long conversations.

Pattern 2 (use ADK's own session service with a custom persistent backend): requires implementing a custom BaseSessionService class, which is significantly more engineering work and tightly couples the persistence layer to the ADK SDK's internal abstractions.

Pattern 3 (minimal JSON column): stores only the fields that are actually needed across turns (turn_count, last_agent, last_ticket_id). The schema is explicit, queryable, and upgrade-safe. The ADK session is a short-lived, per-turn object. This approach requires no external service and survives process restarts because the state lives entirely in SQLite.

6.3. Process Restart Behaviour

When uvicorn restarts:

1. All Python runtime state is wiped. There are no module-level caches or global variables holding session data.
2. The next incoming request creates a new AsyncSession, fetches the sessions row from SQLite, deserializes the state JSON column, and rebuilds the ADK runner with that state injected.
3. The conversation continues as if nothing happened.

This is verifiable in the Loom demo: a multi-turn conversation is started, uvicorn is stopped with Ctrl+C, restarted, and a follow-up message is sent. The server correctly knows the `user_id`, `plan_tier`, and prior `turn_count` without re-asking.

6.4. ADK Event Parsing

After `runner.run_async()` completes, the async event stream is iterated to extract structured information:

```

1 tool_calls = []
2 routed_to = "unknown"
3 retrieved_chunk_ids = []
4 final_reply = ""
5
6 async for event in response:
7     if event.type == "tool_call":
8         tool_calls.append({
9             "tool_name": event.tool_name,
10            "args": event.args,
11            "call_id": event.call_id,
12        })
13     if event.type == "tool_result":
14         # merge result back; extract chunk_ids if this was search_docs
15         for tc in tool_calls:
16             if tc["call_id"] == event.call_id:
17                 tc["result"] = event.result
18                 if tc["tool_name"] == "search_docs":
19                     for r in event.result:
20                         retrieved_chunk_ids.append(r["chunk_id"])
21     if event.is_final_response():
22         routed_to = event.author # sub-agent name
23         final_reply = event.content.parts[0].text

```

7. API Design

7.1. Endpoints

Method	Path	Description
POST	/v1/sessions	Creates a new session. Body: {user_id, plan_tier}. Returns {session_id}.
POST	/v1/chat/{session_id}	Sends a message. Body: {content}. Returns {reply, routed_to, trace_id}.
GET	/v1/traces/{trace_id}	Returns the full trace row as JSON for debugging.
GET	/healthz	Returns {"status": "ok"}. Used by Docker health checks.

7.2. Error Responses

HTTP Code	Error Code	Trigger
404	SESSION_NOT_FOUND	session_id does not exist in DB
404	TRACE_NOT_FOUND	trace_id does not exist in DB
504	UPSTREAM_TIMEOUT	LLM did not respond within LLM_TIMEOUT_SECONDS
422	(Pydantic default)	Request body fails validation

All error responses are JSON objects with at minimum {"error": "<ERROR_CODE>", "detail": "<message>"}.

7.3. Pydantic v2 Models

All request and response bodies use Pydantic v2 models:

```

1 class CreateSessionRequest(BaseModel):
2     user_id: str
3     plan_tier: Literal["free", "pro", "enterprise"]
4
5 class CreateSessionResponse(BaseModel):
6     session_id: str
7
8 class ChatRequest(BaseModel):
9     content: str
10
11 class ChatResponse(BaseModel):
12     reply: str
13     routed_to: str
14     trace_id: str

```

8. Extensions

8.1. E2: Escalation Agent

A third sub-agent was added to the root agent's tool list. When a user message expresses a complaint or requests escalation, the root agent routes to `escalation_agent`, which calls `create_ticket()`. The tool writes a row to the `tickets` table and returns the generated UUID as the ticket ID. The pipeline then stores this in `session.state["last_ticket_id"]` so follow-up turns can reference the ticket.

8.2. E5: Guardrails

Two mechanisms are implemented:

Out-of-scope refusal: Before the ADK runner is invoked, `check_guardrails(user_message)` checks the lowercased message against a keyword list: ["poem", "joke", "story", "write a song", "homework help", "creative writing"]. A match returns a predefined refusal message immediately, with no LLM call and no API cost. Only the user message is logged; no `AgentTrace` is written.

PII redaction: After event parsing but before writing to the DB, `redact_tool_calls()` applies two regular expressions over all tool call arguments and results:

```

EMAIL_RE = re.compile(r'[\w\.-]+@[\w\.-]+\.\w+')
PHONE_RE = re.compile(r'\+?\d[\d\s\-\(\)]{7,}\d')

```

Any match is replaced with [REDACTED]. This prevents PII from entering the `agent_traces` table even if a user inadvertently includes it in a message.

8.3. E6: Docker

The Dockerfile uses `python:3.11-slim` as the base image, installs `uv`, and runs the server on port 8000:

```
FROM python:3.11-slim
RUN pip install uv
COPY . /app
WORKDIR /app
RUN uv sync --no-dev
EXPOSE 8000
CMD ["uv", "run", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--port",
    "8000"]
```

The `docker-compose.yml` mounts two volumes to persist the database and vector store across container restarts: `./helix.db:/app/helix.db` and `./chroma:/app/.chroma`. The `.env` file is passed in via `env_file: .env`.

9. Testing

9.1. Test Suite Overview

`pytest -q` must pass from a clean clone. Three test files cover integration, RAG, and guardrails.

9.2. Integration Test (`tests/test_integration.py`)

The LLM is mocked at the ADK runner boundary using `unittest.mock.AsyncMock`. A `fake_event_stream_generator` yields realistic fake ADK event objects that mimic the shape of a real `knowledge_agent` response.

The test sequence:

1. Create a session via `POST /v1/sessions`.
2. Send a first message. Assert `routed_to == "knowledge_agent"`.
3. Send a second message to the same `session_id`. Assert the response contains a non-empty `trace_id` that is different from the first turn's `trace_id`, proving the session was correctly loaded from DB (rather than raising a `SessionNotFoundError`).

The key assertion is that turn 2 works at all – if state was not persisted after turn 1, the pipeline would fail to find the session and raise a 404. The test also verifies that both trace rows share the same `session_id`.

9.3. RAG Unit Test (`tests/test_rag.py`)

```
1 @pytest.fixture
2 def ephemeral_chroma(monkeypatch):
3     client = chromadb.Client() # in-memory, no disk I/O
4     collection = client.create_collection("helix_docs")
5     collection.add(
6         ids=["chunk_abc123", "chunk_def456", "chunk_ghi789"],
7         documents=["doc1", "doc2", "doc3"],
8         embeddings=[[0.1] * 768, [0.2] * 768, [0.3] * 768],
9     )
10    monkeypatch.setattr("chromadb.PersistentClient", lambda **kw: client)
```

```

11 monkeypatch.setattr(
12     "google.generativeai.embed_content",
13     lambda **kw: {"embedding": [0.1] * 768},
14 )
15 return collection
16
17 async def test_search_docs_returns_valid_scores(ephemeral_chroma):
18     results = await search_docs("rotate deploy key", k=3)
19     assert len(results) > 0
20     for r in results:
21         assert r.chunk_id != ""
22         assert 0.0 <= r.score <= 1.0

```

9.4. Guardrails Test (tests/test_guardrails.py)

```

1 async def test_poem_request_is_refused(async_client, created_session):
2     response = await async_client.post(
3         f"/v1/chat/{created_session}",
4         json={"content": "write me a poem about cats"},
5     )
6     assert response.status_code == 200
7     data = response.json()
8     assert "routed_to" not in data or data["routed_to"] == ""
9     assert "cannot" in data["reply"].lower() or "outside" in data["reply"].lower()

```

10. Security Audit Results

A code audit was run after implementation. No vulnerabilities were found. Key findings:

Attack Vector	Mitigation	Status
SQL injection	SQLAlchemy parameterized queries; no raw SQL strings	Safe
Path traversal in ingest	pathlib.Path.rglob("*.md") never evaluates relative paths	Safe
PII leakage in traces	redact_tool_calls() runs before every DB write	Safe
XSS	FastAPI returns JSON only; no HTML rendering	Safe
API key exposure	.env in .gitignore; .env.example has no real values	Safe
Sync I/O in async handlers	All DB calls use await with AsyncSession	Safe
Bare except: clauses	All caught exceptions are specific types	Safe

11. File Structure Reference

```

helix-srop-assignment/
|-- .env                # local secrets (gitignored)
|-- .env.example        # template with placeholder values
|-- .gitignore
|-- ASSIGNMENT.md       # original assignment spec
|-- README.md           # setup, architecture, decisions
|-- Dockerfile          # E6: slim Python 3.11 image
|-- docker-compose.yml  # E6: api service + volume mounts
|-- pyproject.toml       # all dependencies + ruff/mypy/pytest config
|-- uv.lock
|-- alembic.ini         # untouched (create_all used instead)
|-- alembic/            # untouched
|-- app/
|   |-- __init__.py
|   |-- errors.py       # SessionNotFoundError, UpstreamTimeoutError

```

```
| -- main.py # FastAPI app, lifespan, /healthz, handlers
| -- settings.py # pydantic-settings: keys, URLs, timeouts
| -- agents/
| | -- __init__.py
| | -- account_agent.py # get_recent_builds, get_account_status (mock)
| | -- escalation_agent.py # create_ticket (E2)
| | -- knowledge_agent.py # search_docs tool, citation instruction
| | '-- root_agent.py # srop_root, AgentTool routing
| -- api/
| | -- __init__.py
| | '-- routes.py # /sessions, /chat/{id}, /traces/{id}
| -- db/
| | -- __init__.py
| | -- models.py # Session, Message, AgentTrace, Ticket
| | '-- session.py # async_engine, AsyncSessionLocal, get_db()
| -- obs/
| | -- __init__.py
| | '-- logging.py # setup_logging() via structlog
| -- rag/
| | -- __init__.py
| | -- ingest.py # CLI: chunk, embed, upsert to ChromaDB
| | '-- retriever.py # search_docs(), ChunkResult dataclass
| '-- srop/
| | -- __init__.py
| | '-- pipeline.py # run_turn(): the core orchestration loop
| -- docs/ # 10 Helix Markdown docs (RAG corpus)
| | -- rag-guide.md # reference guide (not ingested as product doc)
| | -- google-adk-guide.md
| | -- fastapi-async-guide.md
| | '-- *.md # Helix product documentation
| -- tests/
| | -- __init__.py
| | -- test_guardrails.py # E5: refusal and PII redaction
| | -- test_integration.py # multi-turn state persistence with mocked ADK
| | '-- test_rag.py # search_docs with ephemeral Chroma + mocked embed
```

12. Setup Instructions

```
1 # Clone and enter the repository
2 git clone https://github.com/vaibhav3000/helix-srop-assignment.git
3 cd helix-srop-assignment
4
5 # Install uv (skip if already installed)
6 pip install uv
7
8 # Install all dependencies
9 uv sync
10
11 # Set up environment variables
12 cp .env.example .env
13 # Edit .env and set GOOGLE_API_KEY=your-api-key-here
14
15 # Populate the vector store (run once before starting the server)
16 uv run python -m app.rag.ingest --path docs/
17
18 # Start the server
19 uv run uvicorn app.main:app --reload
20
21 # Run tests
22 uv run pytest -q
23
24 # Check code quality
```

```
25 uv run ruff check .
26
27 # Or with Docker (Extension E6)
28 docker-compose up --build
```

13. Summary of Design Decisions

Decision	What was chosen	Why
State persistence	Pattern 3 (JSON column)	No external service, survives restart, explicit schema, horizontally scalable
Chunking strategy	Heading-aware + sentence sub-split	Preserves logical sections in Markdown, reduces context fragmentation
ADK routing	AgentTool via function calling	Structured, not string-parsed; required by assignment; robust to rephrasing
ADK session scope	Per-turn transient InMemorySessionService	Simpler than a custom BaseSessionService; state lives in DB not in ADK
Vector store	ChromaDB on-disk	Assignment recommendation; simple setup; upsert prevents re-ingest duplicates
Embedding model	gemini-embedding-2 (768 dims)	Native to Google AI ecosystem; consistent with LLM provider
Test mocking boundary	ADK runner run_async method	Isolates tests from network; fast CI; still tests pipeline logic end-to-end